Transformer Inference Cost Model

Deep Dive into SGLang — Chapter 3

Yin Yang

Foundation Books Project

Where this chapter sits

Chapter 2 reduced serving to an online decision. This chapter gives that decision a cost ledger.

- A decision is only useful if the runtime can estimate what each candidate step *costs*.
- The estimate asks three things: how much accelerator work the step launches, how much attention context it touches, and how much fast memory stays resident afterward.
- Everything later — continuous batching, KV management, attention backends — attaches its constants to *this* ledger.

Goal: read a candidate serving step and write down its work and memory account before deciding cheap, expensive, or infeasible.

Roadmap

The mismatch

The five accounting axes

Physical feasibility

Why it is honest

The mismatch

The dashboard lies: three requests, a big bill

The dashboard says **three active requests**, so the step sounds small.

The ledger says otherwise:

97 scheduled rows · 5,937 attention pairs · 449 live cache positions.

- The dashboard was not wrong — it was looking at the wrong object.
- Request count is blind to work shape and resident state.
- This chapter turns that mismatch into an account the scheduler can use.

The running micro-batch: three requests, one step

One mixed step on `google/gemma-4-E4B-it`:

Request	Current ℓ_r	Prefix p_r	New e_r	Situation
r_{pan}	96	96	32	prefill after full reuse
r_{fresh}	0	0	64	fresh prompt, no reuse
r_{decode}	256	256	1	one-token decode

Scheduled token volume $T(\mathcal{R}) = 32 + 64 + 1 = 97$ — not 3. Post-step lengths become 128, 64, 257. We carry this batch through the whole deck.

The five accounting axes

Five quantities, one serving step

Token volume $T(\mathcal{R})$ newly scheduled rows — $\sum_r e_r$.

Attention pairs $A(\mathcal{R})$ context-read work: scheduled rows times the history they read.

Dense work $D(\mathcal{R})$ per-row projection / MLP work $= T(\mathcal{R})\, C_{\mathrm{dense}}$.

Live capacity K_{live} page-aligned resident slots owed after the step.

Request slots $R_{\max}$ rows in the request table, a *separate* limit.

Compute axes range over the selected step; capacity axes range over the whole post-step live set. They join only at the feasibility test.

The key separation: selected work vs. carried state

- **Dense work** follows *selected rows* only — the prefix is already computed.
- **Attention** reads *carried context* too — reuse shrinks e_r but never the reads.
- **Memory** is owed by every *unfinished* request, even those not executing this step.

Confusing these three is the most common cost-model mistake.

From concept to code: the execution mode

SGLang names prefill / decode / mixed directly, as a forward mode.

from `python/sglang/srt/model_executor/forward_batch_info.py`

```python
class ForwardMode(IntEnum):
    # Extend a sequence. The KV cache of the beginning part of the
    # sequence is already computed (e.g., system prompt).
    # It is also called "prefill" in common terminology.
    EXTEND = auto()
    # Decode one token.
    DECODE = auto()
    # Contains both EXTEND and DECODE when doing chunked prefill.
    MIXED = auto()
```

Phase selects the active cost terms: `EXTEND` is prefill work ($e_r > 1$ common), `DECODE` is $e_r = 1$, and our running batch is `MIXED`.

Physical feasibility

Logical length is not physical cost

Two refinements turn logical lengths into a physical account:

1. **Per-token footprint.** One Gemma cached position is $42 \cdot 2 \cdot (256{+}256) \cdot 2 = 86{,}016$ bytes $= 84$ KiB.
2. **Page alignment.** Capacity is handed out in whole pages of size P; the tail of a sequence wastes *slack*.

With $P = 16$, post-step lengths $128, 64, 257$ allocate $128 + 64 + 272 = 464$ slots — the 257 rounds up to 272, 15 slots of slack.

And a second, independent limit: the request-row table holds at most $R_{\max}$ live requests, regardless of token slots.

The whole step, as one vector

For the running micro-batch:

$$\big(\, T,\ A,\ D,\ K_{\text{live}} \,\big) = \big(\, 97,\ 5937,\ 320{,}392{,}396{,}800,\ 464 \,\big).$$

- $T = 32 + 64 + 1 = 97$ scheduled rows.
- $A = 3600 + 2080 + 257 = 5937$ attention pairs.
- $D = 97 \cdot 3{,}303{,}014{,}400$ MLP-only multiply–accumulates.
- $K_{\text{live}} = 464$ page-aligned resident slots.

Feasible when $K_{\text{live}} \le C_{\text{tokens}}$ *and* live count $\le R_{\text{max}}$. One vector, two accounts, joined at the test.

Model variants refine terms; the discipline is fixed

GQA, MLA, NSA, SWA, FP4 each change *one* part of the account:

Variant	What it refines
Grouped KV heads (GQA)	bytes per resident token (fewer KV heads)
Latent cache (MLA)	resident payload $\rightarrow$ latent + RoPE dims
Sparse index (NSA)	adds indexer buffers
Sliding window (SWA)	splits capacity across pools
FP4 cache	shrinks payload, adds scale metadata

Rule: a variant may *refine* a term, never silently *delete* one. Counting only payload bytes while dropping scale or index buffers violates the model-shape invariant.

Why it is honest

The model is a cost vector, not a stopwatch

- Changing a cost-axis input moves the measured surface *in the direction that axis names* — not a one-to-one latency law.
- Jetson Thor sweep: raising decode context $512 \to 8192$ grows $A(\mathcal{R})$ about $16\times$, but median ITL grows only $\approx 1.9\times$.
- Fixed overheads (scheduling, launch, sampling, dense rows) absorb most of the context-read growth at that scale.

Capacity scales exactly with the ledger; elapsed time needs a calibrated fixed term and a backend slope.

What to carry forward

1. A serving step is a **cost vector**: token volume, attention pairs, dense work, resident capacity, request slots.
2. **Selected work** and **carried state** are different accounts — never collapse them.
3. **Prefix reuse** shrinks e_r and dense work, but reused context still appears in attention reads.
4. Feasibility is **physical**: page-aligned slots and request rows, not raw sequence length.
5. Every term has a **home in SGLang source** — `ForwardBatch`, pool configurators, the allocator.

Next: continuous batching, chunked prefill, and KV management all make policy choices from this same ledger.